

BATTLE – Bonded Adversarial TournamenT with Logarithmic Escalation

Sergio Demian Lerner* ^{1,2} and Ariel Futoransky† ¹

¹Fairgate Labs

²Rootstock Labs

Abstract

In our work, we introduce BATTLE, Bonded Adversarial TournamenT with Logarithmic Escalation, a tournament-style protocol that solves multiparty disputes with simultaneous assertions such that (i) bounds honest asserter capital requirements to a constant minimum initial capital (A_{MIC}) and (ii) resolves any number C of concurrent challenges in $\mathcal{O}(\log C)$ dispute rounds, by reinvesting dispute rewards to fund subsequent rounds (progressive buy-ins) (iii) can be realized on a stateful (Quasi)Turing-complete smart-contract enabled blockchain. BATTLE solves a set of conflicting assertions by creating a tournament with two phases: (1) a bracket among competing asserters with one dispute per party per round, and (2) a challenger phase against the winning assertion where the assenter engages in increasing number of simultaneous disputes each round.

1 Introduction

Public blockchains face a well-known “scalability trilemma”: improving any two of decentralization, security, and throughput tends to degrade the third. Raising the base layer’s capacity (larger blocks, faster finality, more complex on-chain execution) typically concentrates validation or widens the attack surface. Conversely, preserving permissionless validation and strong security caps on-chain transaction throughput.

Current practice addresses this constraint by moving execution off the base layer and interconnecting multiple domains via bridges. Layer 2 systems (rollups, channels, validity/fraud-proved VMs) batch or prove large volumes of transactions while relying on an L1 for data availability and settlement; parallel L1s specialize for different use cases and security budgets. Bridges knit these domains together, enabling horizontal scaling without inflating L1 consensus. This approach preserves the security and decentralization of the settlement layer while increasing aggregate throughput across domains. Bridge viability, however, hinges on the correctness of cross-domain claims, its promptness and decentralization.

Optimistic bridges default-accept cross-chain claims subject to a challenge period, yielding high throughput and low on-chain verification cost while preserving permissionless participation and safety under the “at least one honest challenger” assumption—thus avoiding trusted relays and accommodating general-purpose state proofs.

Optimistic bridges require an on-chain mechanism to resolve disputes efficiently. We abstract away asset-custody mechanics (locking, burning, and their proofs) and isolate dispute resolution as a standalone problem. We formalize this problem by specifying the interface and guarantees of a protocol

*sergio@fairgate.io

†futo@fairgate.io

that adjudicates concurrent claims; we call this abstraction the *Dispute Resolution of Concurrent Assertions* (DROCA). In DROCA, parties submit assertions about the output of a fixed computation on given inputs, and other parties may challenge those assertions. Assertions are partially ordered by a parent relation: a child depends on the correctness of its parent, and falsifying a parent invalidates all descendants. A smart contract mediates acceptance or rejection under adversarial concurrency. Although dispute-resolution protocols address varied requirements, we say that a concrete protocol qualifies as a DROCA protocol if it admits an instantiation that implements the DROCA interface and satisfies its guarantees.

DROCA protocols are needed in settings that lack a native one-to-many adjudication primitive and provide only a two-party dispute primitive (a “verification game” [1, 2]). A DROCA protocol orchestrates and schedules multiple two-party disputes to resolve the set of concurrent assertions across all parties.

DROCA protocols can be found in several optimistic blockchain systems: rollups [3], bridges [4–6] and off-chain delegated computation [1, 7]. In fact, the most important security properties of these systems depend on the DROCA they use. For example, on a token bridge, the DROCA protocol is the main contributor to the promptness, safety, and decentralization of the system. Due to the proliferation of optimistic blockchain systems, there is widespread interest in new more efficient DROCA designs.

The hardest problem related to DROCA protocols is DoS resistance. Defending an assertion in a dispute has a cost for each participant, mostly in on-chain transaction fees, but there is also a financial cost of temporarily locked security bonds. If dispute initiation is permissionless and multiple disputes against an honest prover can occur concurrently, then a Sybil attacker may overwhelm the prover by initiating a large number of simultaneous challenges, potentially exhausting the prover’s resources and causing them to lose disputes.

The opposite can also occur; a large number of malicious provers can overwhelm the capacity of an honest challenger to challenge them all. Recent research on this topic includes the PRT [8], BoLD [9], Dave [10] and Optimism [11] protocols. These protocols run on blockchains with Turing-complete stateful smart contracts such as Ethereum [12] or Rootstock [13].

1.1 Protecting Asserters Against DoS Attacks

To protect honest challengers from avalanches of assertions and protect honest asserters from avalanches of challenges, protocols bound dispute concurrency.

However, even with concurrency limits, worst-case cumulative dispute expenditures can remain large and exert centralization pressure. Under sustained challenge load, liquidity-constrained provers may be unable to mobilize capital or obtain short-term credit at the required pace to open and complete all disputes, increasing the risk of default; in practice, participation remains viable primarily for well-capitalized provers.

A natural response is to serialize disputes so they do not overlap. With C challengers, this yields a makespan linear in C , which is impractical at scale. When reimbursement of fronted funds is gated on dispute resolution, prolonged sequences keep the operator’s capital locked and accrue financing or opportunity costs that may exceed available liquidity. Consequently, purely sequential scheduling can be as problematic as fully concurrent execution; these are opposite extremes of the design space, each inducing unacceptable operational risk.

The search for an arrangement of disputes that is optimal under capital and latency constraints has motivated new multiparty dispute resolution protocols. These protocols structure interactions by (i) pairing proponents with opponents, (ii) allowing the proponent to confront opponents in batches across multiple dispute rounds, and (iii) enabling parties that defend the same position to share parts of their

interactions [10]. We refer to these structured designs collectively as tournaments.

1.2 Contributions

Existing tournament-based dispute resolution protocols employ slashing to forfeit the security bonds of losing parties, but they generally omit progressive reward accounting. In particular, the proceeds from a successful dispute can be redeployed to finance subsequent on-demand bonds and fees in later rounds of the same tournament. We make this reuse explicit and incorporate it into the economic model, treating dispute rewards as escrowed liquidity for future engagements rather than terminal payouts.

Progressive rewards are rare in real-world tournaments. In certain online poker tournaments or sit-and-go games, players need to pay an entry fee to participate, and sometimes additional fees are required as the game progresses (e.g., to continue in higher stakes rounds). Players may need to use their winnings from earlier rounds to buy into later rounds or higher stakes tables. Also in Progressive Knockout (PKO) games, players receive a bounty after each game is won.

In this work, we apply the PKO and progressive buy-in concepts to DROCA protocols to present BATTLE – Bonded Adversarial Tournament with Logarithmic Escalation, and describe how it can be implemented on top of a stateful (Quasi)Turing-complete smart-contract platform.

BATTLE is a DROCA protocol that achieves logarithmic-time dispute resolution while keeping the honest asserter’s on-demand security bonds constant. The protocol proceeds in rounds: in round r it opens k_r disputes, with a non-decreasing schedule $k_{r+1} \geq k_r$ (typically $k_{r+1} \approx 2k_r$). Dispute rewards are paid before the next round and are earmarked to fund the security bonds and fees of round $r + 1$, enabling exponential growth in per-round concurrency and thus $\mathcal{O}(\log C)$ rounds to resolve C challengers.

2 DROCA Protocols

We define a DROCA protocol, adopting a framework inspired by [9], which not only models the basic multiparty dispute problem, but also considers the existence of multiple conflicting assertions and child assertions. Other formalizations [2] do not model multiple concurrent assertions in conflict.

The purpose of a DROCA protocol is to allow several parties to resolve conflicting assertions. Honest participants try to defend the true assertions and challenge the false assertions, so that the former are accepted, and the latter are rejected.

There is a universe S of potential assertions. Each assertion $s \in S$ is a tuple $s = (p, x, o)$ where $p \in S \cup \{S_0\}$ is a reference to its parent assertion, $x \in \{0, 1\}^*$ encodes the arguments as an opaque byte string, and $o \in \mathcal{O}$ is an actionable outcome that determines the contract’s behavior if s is accepted.

Example 2.1. *In a DROCA that verifies optimistic rollup state transitions on L1, an assertion $s = (p, x, o)$ is instantiated as follows: p is the identifier of the last accepted L2 state; x is a SNARK (with its public inputs) proving that applying a batch of L2 transactions to p yields a new state p' ; and o encodes the set of L1 withdrawals implied by that batch. The assertion verification program accepts s if and only if the proof in x verifies and the withdrawals in o are consistent with p' .*

Example 2.2. *In a DROCA that verifies optimistic bridge peg-outs, an assertion $s = (p, x, o)$ is instantiated as follows: p identifies the last accepted peg-out; x is a SNARK (with public inputs) proving that the sidechain emitted a canonical event attesting a burn of T tokens destined to mainchain address D ; and $o = (T, D)$ encodes the actionable payout on the mainchain. The assertion verification program accepts s if and only if the proof in x verifies and the event’s contents match o while respecting the ordering implied by p .*

The protocol specifies an *Assertion Verification Predicate* (AVP) that decides the correctness of assertions within a bounded number of execution steps. Formally, let θ denote contract-fixed parameters (e.g., authorized asserter set, last verified block timestamp). The predicate

$$\text{AVP} : S \times \Theta \rightarrow \{0, 1\}, \quad \text{AVP}((p, x, o); \theta) \in \{0, 1\}$$

returns 1 if and only if the assertion $s = (p, x, o)$ is correct under θ , and 0 otherwise. The AVP must be locally computable by any party within time at most T_{avp} , a protocol constant. The assertion s included among the predicate inputs. Some inputs to the predicate may be fixed by the contract (e.g., asserter identity, assertion timestamp, or block height) and folded into θ . When time-dependent inputs are included, the AVP should be *nonincreasing in time*: if $\text{AVP}(s, \theta, t) = \text{false}$ at some time t , then $\text{AVP}(s, \theta, t')$ must remain **false** for all $t' \geq t$. Equivalently, AVP may transition **true** $\rightarrow$ **false** (e.g., upon expiry) but must never transition **false** $\rightarrow$ **true** solely due to the passage of time. This prevents strategic waiting from converting an invalid assertion into a valid one.

Asserters can evaluate AVP on their own candidate assertions off-chain prior to communication, so the predicate’s outcome is known to them before submission. Last, the system must define a blockchain-based two-party verification game that can convince a contract of the validity of the predicate.

A DROCA smart contract receives assertions $s = (p, x, o)$, records their status (accepted/rejected/unresolved), and enforces the parent–child consistency implied by p . The outcome o specifies the action the contract executes upon acceptance. The argument x may contain a correctness proof (e.g., a SNARK [14]); in that case the assertion verification predicate AVP is instantiated as the SNARK verification algorithm for a contract-fixed verification key vk .

Let $\mathcal{P}$ denote the set of parties that interact with the DROCA contract. The contract parameterizes two roles: *asserters* $\mathcal{A} \subseteq \mathcal{P}$, who are authorized to submit assertions and defend them in disputes, and *watchtowers* $\mathcal{W} \subseteq \mathcal{P}$, who are authorized to register and prosecute challenges. Parties may hold both roles. Parties are also known as *operators*. The admissible role sets (permissioned or permissionless) are part of the contract parameters θ .

Each assertion maintains a state $\sigma(s) \in \{\text{accepted}, \text{rejected}, \text{unresolved}\}$. If an assertion $s = (p, x, o)$ references a parent p , then s presupposes the correctness of p and may be accepted only if p is (already) accepted. The genesis assertion S_0 is assumed correct and is accepted ab initio. Any party may post an assertion $s \in S$ provided that its parent has been posted. We define the *sequence number* (depth) of s as $\text{depth}(s)$, the length of the parent chain from S_0 to s .

Two asserters may submit *equivalent* assertions. Formally, define an equivalence relation $\sim$ on S by

$$s = (p, x, o) \sim s' = (p', x', o') \iff \text{AVP}(s) = \text{AVP}(s') = 1 \text{ and } o = o'.$$

Equivalence does not require identical messages: for example, two parties may attest to the same token transfer using distinct SNARKs in x and x' , owing to the non-determinism of proof generation, while inducing the same outcome $o = o'$. A rational challenger should not dispute a correct assertion, even if it is merely equivalent to a previously posted correct assertion, since the challenge will be lost under the AVP.

We distinguish two kinds of assertion conflict. Let $\preceq$ denote the reflexive–transitive closure of the parent relation (“inherits from”), and let $\text{depth}(\cdot)$ be the sequence number.

Sibling conflict: two distinct, non-equivalent assertions s_A, s_B with $\text{depth}(s_A) = \text{depth}(s_B)$.

Relative conflict: two distinct, non-equivalent assertions s_A, s_B such that $\text{depth}(s_A) \neq \text{depth}(s_B)$ and $s_A \not\preceq s_B$ and $s_B \not\preceq s_A$ (i.e., they are incomparable in the ancestor partial order).

All assertions $s \neq S_0$ are initially $\sigma(s) = \text{unresolved}$, while $\sigma(S_0) = \text{accepted}$. If two non-equivalent

assertions s_A, s_B are in conflict (sibling or relative), the contract enforces mutual exclusivity:

$$(s_A \text{ conflicts with } s_B) \implies \neg(\sigma(s_A) = \text{accepted} \wedge \sigma(s_B) = \text{accepted}).$$

Let $\text{auth} : S \rightarrow \mathcal{A}$ denote the asserter that posted an assertion. An asserter u is prohibited from submitting duplicate claims: there do not exist $s \neq s'$ with $\text{auth}(s) = \text{auth}(s') = u$ such that either $s \sim s'$ or s conflicts with s' . Any watchtower may challenge an unresolved assertion, and the assertion's author is obligated to defend it in a dispute. In the base model, each dispute is a two-party protocol between the asserter and a single challenger; extensions that allow coalitions are possible but are not assumed here.

To reject an assertion, a challenger must open a dispute and win it before the assertion's challenge window expires. If a challenger prevails in time, the contract marks the assertion as *rejected*; otherwise, once the window expires, rejection on that basis is no longer possible.

An assertion is marked *accepted* if its challenge window expires without a successful challenge. Upon acceptance, all assertions that conflict with it are automatically marked *rejected*. The contract enforces that any assertion set to *rejected* must have been *unresolved* immediately prior to rejection (no accepted-to-rejected transitions). Assertions equivalent to the accepted one, as well as their descendants, are rejected to preserve uniqueness of outcome, but their asserters are not penalized.

Figure 1 shows a sample DROCA protocol run with 4 assertions and 2 challenges.

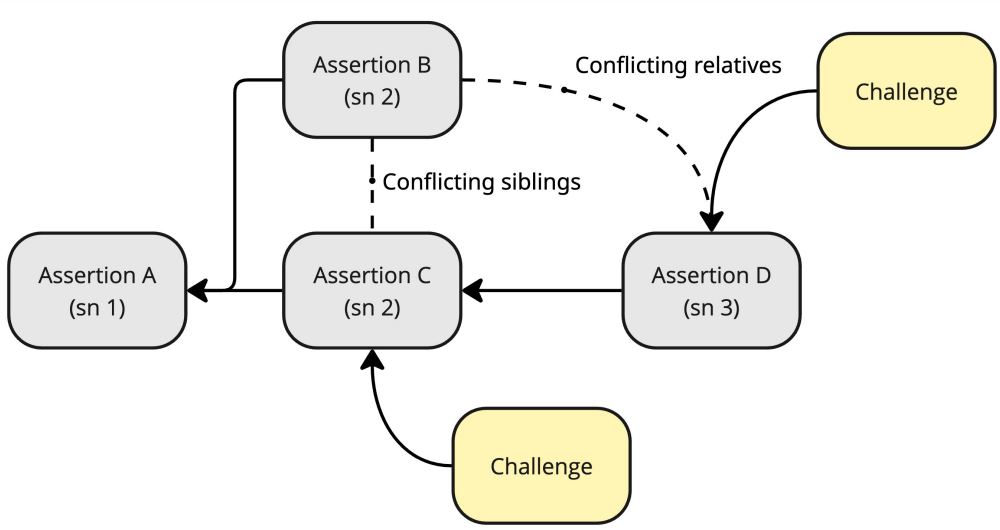


Figure 1: A sample DROCA protocol run

A DROCA protocol comprises multiple disputes. A *dispute* is an interactive two-party protocol between an asserter and a challenger that leverages the L1 chain as arbiter, with the following interface:

1. Each move is submitted to an L1 smart contract as a transaction input.
2. The contract validates moves, enforces timeouts, and eventually determines a winner.
3. Upon resolution, the contract sets the assertion's state to *accepted* or *rejected*.

Participation in DROCA consumes three resource classes:

1. *staking*—capital posted as security bonds, as specified by the protocol;
2. *gas*—L1 currency to pay inclusion fees for assertions and dispute moves;

3. *computation*—off-chain compute (and bandwidth) incurred by participating parties.

These resources may be committed *in advance* (persistent deposits) or *on demand* immediately before or during a dispute. In bridge deployments spanning two blockchains, staking and fee payments may be required on one or on both chains, depending on the contract design.

As for staking, the dispute resolution protocol specifies exactly how much and when stakes must be made. When the smart contract declares a winner, some stakes will be confiscated (“slashed”) and some will be returned to the staking parties: corrupt parties may have some or all of their stake confiscated, while honest parties should get all of their stake returned to them. The staking requirement serves to discourage malicious behavior. In addition, confiscated stakes may be redistributed to honest parties to cover their L1 gas costs and off-chain compute costs, or simply as a reward for participating in the protocol. These stakes will be held in escrow by the smart contract.

We make the following assumptions:

1. L1 provides both liveness and safety, that is, every transaction submitted to it is eventually processed and is processed correctly;
2. at least one honest party participates in the dispute resolution protocol.

2.1 Security Properties

We consider a standard network model that concerns a set of N parties, some of which may be ‘corrupted’ and controlled by an adversary. Some of the parties are asserters and some are challengers, and these sets may overlap. We assume that the network includes a ‘blockchain’, which tracks the amount of digital currency owned by each player. The protocol begins with a set of assertions posted by a subset of asserters, and all the assertions posted are in conflict. Each assertion specifies an actionable outcome.

A protocol implementing N -party DROCA should satisfy the following security properties. Let T_{opt} denote the optimistic (all-honest) time bound and let $T_{\text{pes}} \geq T_{\text{opt}}$ denote the pessimistic (at-least-one-honest) time bound.

1. **Optimistic termination.** If all parties are honest, the protocol terminates by time T_{opt} .
2. **Pessimistic termination.** If at least one party is honest, the protocol terminates by time T_{pes} .
3. **Safety (uniqueness of outcome).** Upon termination, either no assertion is accepted or exactly one assertion is accepted; in the latter case, its outcome is executed.
4. **Collateral soundness.** Any collateral (deposits/bonds) posted by honest parties is returned to them according to the protocol’s rules.
5. **Accountability (penalization).** Parties that post invalid assertions or challenge valid assertions forfeit the corresponding collateral, which is slashed upon resolution.
6. **Incentivization (reward).** If the execution exceeds T_{opt} due to disputes, then at least one honest party receives a reward funded from the collateral of dishonest parties.

The properties above are only required to hold, except for negligible probability. In particular, the protocol relies on cryptography with a security parameter 1^λ , so ‘negligible probability’ here means a negligible function of λ , for example $\text{negl}(\lambda) = 2^{-\lambda}$.

We add two additional properties that target incentive compatibility and efficiency at large N :

- **(Refund).** For each honest party, the protocol must ensure that the resources invested in the execution —namely local computation, working memory, communication bandwidth, and transaction fees— are refunded or offset so that the *net* cost borne by that party is sublinear in the number of parties N .
- **(Fair across repeated executions).** When the protocol is executed multiple times with the same parties, the mechanism that assigns resource-bearing actions must be *ex ante* symmetric among honest challengers and *ex post* balanced. In particular, across any sequence of executions, each honest challenger should bear a share of the residual (unrefunded) costs—computation, memory, bandwidth, and transaction fees—that is comparable to that borne by other honest challengers, allowing at most constant-factor differences and, where unavoidable, only polylogarithmic slack in N . The mechanism that allocates rewards (including bounties or proceeds from slashed collateral) across repeated executions must satisfy the same *ex ante* symmetry and *ex post* balance.

From a design perspective, the minimum initial capital required of honest parties should scale *sublinearly* with the system size N (e.g., $\mathcal{O}(\log N)$). In practice, an $\mathcal{O}(N)$ requirement with small constant factors may be tolerable for moderate N , whereas achieving *constant* capital $\mathcal{O}(1)$ remains the optimal target.

The fairness property implies that the protocol must prevent systematic steering or bias that would repeatedly concentrate costs on a fixed honest challenger orasserter, possibly by including an explicit load-balancing rule (e.g., rotation, randomization, or equivalent) that distributes these costs fairly over time.

2.2 Protocol Attacks

A secure DROCA protocol must be safe from the following types of attacks:

Censorship Attacks. We assume that Layer 1 (L1) maintains both liveness and safety; however, it is susceptible to potential intermittent censorship attacks. An adversary may temporarily prevent transactions submitted by honest parties from being included in L1, while still allowing its own transactions to be processed.

Ordering Attacks. In scenarios where the adversary is not actively censoring transactions, we assume that it may still influence the order of transactions posted to L1. Specifically, the adversary might prioritize its own transactions over those submitted by honest participants within the same L1 block.

Resource Exhaustion Attacks. Although protocols may be designed to ensure that honest parties are compensated after successful completion, the adversary can attempt to deplete the critical resources of honest parties, such as staking, gas, or computational resources. This depletion could cause honest parties to be unable to continue participating in the protocol.

Delay Attacks. The adversary may attempt to delay the resolution of disputes beyond a reasonable time frame. This could involve prolonging the protocol’s execution without a resolution, which might eventually lead to a resource-exhaustion attack, as the honest parties are unable to sustain the prolonged dispute.

The principal adversarial vector in DROCA is denial-of-service via Sybil-driven challenge flooding and resource exhaustion. In the next section, we survey DoS mitigation techniques.

2.3 Avoiding Denial of Service Attacks

In this section, we discuss the different methods that DROCA protocols can use to reduce the impact of denial-of-service attacks.

2.3.1 Computation History Commitments

To reduce challenger effort, challengers opposing the same claim can collaborate; likewise, asserters defending the same assertion may delegate to a single representative. However, in many two-party protocols (e.g., BitVMX), a malicious asserter of a *correct* assertion s can force its rejection by self-sabotaging during the dispute, even without an accomplice. Consequently, an honest party cannot rely on another asserter’s submission of s (or an equivalent s') and must assert and defend s personally.

In contrast, protocols such as Dave and BoLD employ *computation history commitments*: the asserter first commits to all potential future responses, and so does the challenger. The commitment is the root of an authenticated tree of intermediate states; each response in the game corresponds to a path in this tree. Because deviations would contradict the initial commitment, neither side can self-sabotage, and parties that share the same position can safely join forces.

2.3.2 Limiting Equivalent Assertions

Equivalent assertions are problematic: they should not be placed in competition. Only when computation-history commitments guarantee that both parties can safely coalesce behind a single committed assertion should equivalent submissions be admitted. A practical remedy is to authorize exactly one asserter per admissible assertion and reject duplicates at the AVP layer. Concretely, the AVP includes an authorization check binding the assertion to its designated asserter; any submission by a different party fails immediately.

For example, in a typical BitVM bridge, only the liquidity provider is authorized to submit the fronting proof and request reimbursement.

2.3.3 Limiting Sibling Assertions

The *assertion space* is generally large: when the AVP input includes a SNARK, an assertion may span hundreds of bytes. By contrast, the *outcome space* can range from large to minimal. For instance, if the outcome is a tuple (address, amount) specifying a payment, the outcome space is large. In UTXO-style contracts, the outcome may be only a Boolean (accept/reject a specific UTXO spend), yielding a minimal outcome space.

The advantage of a reduced outcome space is that it bounds the number of simultaneous sibling disputes. This reduction is feasible when the contract can cheaply filter invalid assertions. For example, if the outcome carries a liquidity-provider index as a 32-bit integer but only indices in $[1..10]$ are valid, any out-of-bounds index is rejected immediately. Thus, even with thousands of asserters and challengers, the number of active sibling assertions is capped at 10, easing the load on challengers.

If the outcome space is large and the set of asserters is unbounded, a DROCA protocol cannot ensure that an honest challenger can reject all incorrect assertions with fixed initial capital: asynchronously posted sibling conflicts must be contested separately to avoid default acceptance, and the challenger cannot rely on others joining. Hence, the number of unresolved sibling conflicts must be capped—by shrinking the outcome space, limiting who may post assertions, or restricting the assertion creation

rate. This cap applies only to asserter-side concurrency; the number of challengers (watchtowers) remains unrestricted.

2.3.4 Limiting Child Assertions

To keep the number of active assertions manageable, the protocol should bound the *child assertion growth* per branch via a *Maximum Growth Rate* (MGR): no branch may advance faster than this rate, preventing watchtowers from being overwhelmed. In practice, it can be simpler to enforce a *Global Assertion Rate* (GAR) that caps the aggregate creation rate of both sibling and child assertions. GAR has the benefit of adapting automatically: when sibling conflict is low, growth can be higher; when many siblings are created, growth is throttled.

Because the intended MGR/GAR are typically far below the raw transaction or block rate, they are set by the contract rather than relying on L1 throughput limits. If the system admits a purely optimistic path (all parties agree without opening disputes), MGR/GAR should remain inactive until some party defects. In designs that include a challenger decision window prior to opening disputes, the MGR must be substantially reduced as soon as the first dispute is opened.

2.4 Application to Optimistic Bridges

Optimistic bridges [4] rely on a DROCA protocol to adjudicate peg-outs. Because peg-outs form an ordered stream, each peg-out assertion references the previously accepted peg-out as its parent, while disputes on child peg-outs may temporally overlap with those of their parents. To prevent adversarial backlogs, the system enforces a bound on the *maximum growth rate* (MGR) of child disputes (Appendix 2.3), which in turn requires rate limiting of peg-out requests at the bridge interface.

Since peg-outs can be of variable amount, the cumulative peg-out amount rate (a.k.a., monetary flow rate) is not necessarily limited. However, for security reasons, bridge protocols usually limit the monetary flow rate because at a certain threshold it becomes rational to an attacker to attempt to reorganize the blockchain and double-spend peg-outs. A *peg-out* assertion must attest all of the following:

1. **Request and destination.** A user requested a transfer of T tokens from the secondary chain to a designated primary-chain address D .
2. **Acceptance and burn on the secondary chain.** An authoritative contract on the secondary chain accepted the request and effected the burn of T tokens. This is evidenced by an emitted event recorded in an authenticated log (e.g., a Merkle-logged receipt).
3. **Inclusion in a canonical secondary chain.** The event is included in a valid secondary-chain history. For proof-of-work systems, validity is demonstrated by sufficient cumulative work extending beyond the event. For proof-of-stake or proof-of-authority systems, validity is demonstrated by a pre-established quorum of validator confirmations (or an aggregated signature) on the confirming blocks.
4. **Anchoring.** The secondary-chain history is rooted in an agreed anchor (e.g., a checkpoint or genesis block) recognized by the bridge.

In permissioned blockchains, validity typically coincides with canonicity: the approved chain is the canonical chain. In most public blockchains, by contrast, canonicity must be derived by applying a fork-choice rule to competing valid histories (e.g., longest-chain or highest-weight selection).

The per-dispute cost together with the required security bond jointly constrain peg-out throughput and decentralization. Larger security bonds raise the liquidity threshold for participation and thus concentrate operators; they may also support higher per-round dispute concurrency, shortening time-to-finality. Conversely, bonds that are too small expose honest provers to challenge flooding that can exhaust fees and liquidity. System design therefore navigates a three-way trade-off among promptness, safety, and decentralization [15]. In protocols such as BoLD [9], parties post persistent bonds when asserting or counter-asserting (e.g., “peg-out occurred” vs. “did not occur”), and these bonds remain locked for the duration of the challenge window.

3 Prior Work in Rollups

The design of optimistic rollups has traditionally relied on the assumption that fixed-duration challenge periods, secured by any single honest participant, are sufficient to ensure correctness. This assumption led to a shift in focus toward incentive compatibility problems such as the Verifier’s Dilemma [16], wherein the lack of invalid state submissions discourages verifiers from engaging, thereby weakening the protocol’s robustness.

However, recent work has called into question the viability of such assumptions under adversarial conditions. The main blocker is now considered to be resource exhaustion attacks, whereby a well-funded adversary can outlast honest challengers economically, compromising the "1-out-of-N" security assumption. This scenario has led to the emergence of what is now referred to as the fraud proof trilemma [15], which posits a fundamental trade-off between safety, promptness, and decentralization.

Fraud proof systems can be classified along two axes: single-round vs. multi-round and on-chain vs. off-chain. Single-round systems re-execute the full L2 state transition on-chain and were attempted by early designs. However, full re-execution on-chain proved inefficient for general-purpose VMs.

To reduce costs, multi-round fraud proofs rely on interactive protocols to pinpoint a single disputed instruction via bisection games. This strategy is the basis for the fraud proof protocols in Arbitrum [9], Cartesi [10], and Optimism [11]. Each of these systems deploys a distinct virtual machine for on-chain execution of disputed steps (WASM, RISC-V, MIPS, respectively), and implements either full or partial concurrency to manage the trade-off between promptness and capital efficiency.

Arbitrum Classic [17] uses sequential proposals and partial concurrency, ensuring high safety at the cost of increased latency and vulnerability to delay attacks. To mitigate these delays, Arbitrum maintains a whitelist of trusted validators, thereby sacrificing decentralization. In contrast, Arbitrum BoLD (Bisection Optimistic Layered Dispute) [9] introduces computation history commitments and a hierarchical protocol that supports full concurrency and improves capital efficiency. BoLD reduces vulnerability to invalid bisections by enforcing commitment to intermediate states, although at the cost of high computational complexity and large bond requirements (3600 ETH for an asserter).

Cartesi’s "Dave" protocol [10] proposes a novel tournament-based partial concurrency scheme that organizes disputes as refereed bracket tournaments. This design significantly improves capital efficiency by requiring logarithmic resources from honest parties. Like BoLD, Cartesi also employs computation history commitments, but organizes the challenge in stages where state checkpoints are created at recursively greater granularity at each stage, although its partial concurrency model introduces longer potential delays. Its use of low bond requirements (e.g., 1 ETH) enhances decentralization.

Optimism’s fraud proof protocol (OPFP [11]), launched on mainnet in June 2024, avoids history commitments and instead secures correctness through recursive counter-claims and bisection graphs. OPFP supports full concurrency and fixed-delay challenge resolution but requires staking bonds at each depth of the challenge tree, leading to exponential cost under resource exhaustion attacks. Despite its minimal overhead for claim submissions, the bond escalation model leads to an unfavorable resource

ratio, favoring attackers.

Overall, while recent designs like BoLD and Cartesi achieve significant improvements over early optimistic rollup architectures, the fraud-proof trilemma remains unresolved. No existing protocol simultaneously achieves low resource requirements, low settlement delays, and high decentralization. Instead, these systems explore different points along this design spectrum. In particular, no system guarantees constant security bonds for asserters under unbounded challengers that settle in logarithmic time.

None of these protocols works on Bitcoin.

4 BATTLE – Bonded Adversarial Tournament with Logarithmic Escalation

We present BATTLE – *Bonded Adversarial Tournament with Logarithmic Escalation* – a new yet simple multiparty dispute protocol that achieves a satisfactory trade-off between round complexity and cost. In the BATTLE protocol, an honest operator engages in disputes in a form of tournament, where disputes are fought in sequential rounds. When the assenter presents a new assertion, challengers are given a period to register as challengers. In addition, any other assenter having a conflicting claim can present their claims. The tournament starts when the registration period is over. The tournament has two phases. In the first, the asserters of conflicting claims fight each other until one assertion survives, and in the second phase, the assenter defending the surviving assertion engages in disputes with all the challengers. The tournament ends when a challenger wins or all challengers lose and a single assenter wins.

Time will be split into discrete epochs (or rounds). We also use the term *timelock period* to refer to a period long enough that transactions cannot be censored, ensuring blockchain inclusion. An interactive round may consist of several timelock periods.

Although assertions can occur at any time, they are queued in an inactive state and only activated at the beginning of an epoch. Child assertions are queued and kept inactive until parent assertions are resolved. All disputes can be resolved within an epoch. The schedules are created so that every unresolved active assertion is disputed on consecutive epochs without gaps. There can be more than one unresolved assertion disputed in parallel. In addition, at each tournament round, multiple disputes can be held for the same assertion. Dispute rewards are paid before the end of an epoch, so they are readily available to be used at the beginning of the next.

From now on, let C be the number of challengers and N the number of operators (who can be asserters or challengers).

4.1 Assumptions

To protect BATTLE against Censorship, Ordering, Resource Exhaustion, and Delay Attacks, we need to make some assumptions. We treat disputes as black boxes: we do not analyze how individual disputes are resolved. We assume that 'chess clock' timeouts ensure that a dispute cannot take more than a bounded amount of time. Therefore, we abstract the continuous nature of time and assume that time is divided into discrete epochs, all disputes can be fully resolved in one epoch (or 'round').

When no chess clock is used, we assume that individual transaction timelock periods within the epoch are chosen so that censorship attacks are irrational: spamming the blockchain to exclude a transaction will cost more to the attacker than the value at stake to grab. Therefore, we assume that no delay attack will be sustained for a timelock period or more. We do consider the incidence of delay attacks

of periods below a timelock period, and so we show that our protocol is free from race conditions that can emerge from short delays.

We assume that the issuance of dispute transactions does not generate network or block congestion. Therefore, the disputes do not interfere with each other at the network level, nor compete for block space or block gas.

We assume that honest parties can follow the protocol and act immediately when they need to. If a party needs to perform a local computation before issuing a transaction, then the global timelock period or chess clock is adjusted to account for the worst-case local computation time, added to the censorship resistance requirement.

We assume that when there is no ongoing spamming attack, transactions are immediately included in the blockchain (the time it takes to the first confirmation is considered negligible compared to the timelock period). We assume that funds received can be spent immediately without waiting for block confirmations after the transaction is included in the blockchain.

We assume that disputes do not carry extractable value to block producers. Nevertheless, our protocol is immune to transaction ordering attacks within the same block unless it happens in the boundary of a timelock period.

To summarize, we assume parties do not delay their responses, local computation is accounted for in the time-lock period, and transactions are included in blocks immediately when they are broadcast. Unexpected transaction delays are entirely attributed to censorship attacks.

We care especially about resource exhaustion attacks against asserters and set our initial capital requirements so that the resources of honest parties cannot be exhausted.

4.2 Protocol

A *full tournament* consists of two *partial tournaments*—Phase 1 and Phase 2—and resolves all assertions at the next unresolved sequence number.

When Phase 1 begins, an assenter-registration window of one timelock period is opened. If only one assertion is registered, Phase 1 is skipped. When Phase 2 begins, a challenger-registration window of one timelock period is opened. If no challenger registers, the surviving assertion is accepted at the close of that window.

The full tournament for a given sequence number comprises (i) all disputes among sibling assertions collected during the Phase 1 registration window and (ii) all disputes between the surviving assenter and the challengers registered in Phase 2. Figure 2 illustrates the registration flow for asserters and challengers.

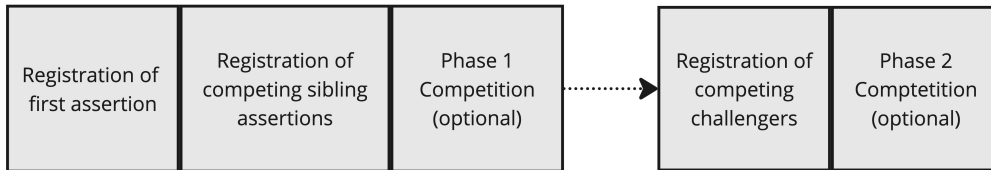


Figure 2: The assenter/challenger registration process

In Phase 1, each assenter is guaranteed exactly one dispute per round. In Phase 2, the surviving assenter engages each round in a number of disputes that is non-decreasing relative to the previous

round (except possibly in the final round, where the remaining opponents may be fewer than the available dispute slots).

If a tournament at sequence number q is active, no new assertion with sequence number q may be posted. Upon termination:

- If some assertion s with $\text{depth}(s) = q$ is accepted, the result is *final*: no future assertion with sequence number $\leq q$ is admissible, and all subsequent assertions must inherit from s (i.e., $s \preceq s'$ for any new s').
- If all assertions in the tournament are rejected, the slot at sequence number q re-opens and new assertions with sequence number q may be posted.

4.3 Setup

The protocol begins with a registration phase in which each party $u \in \mathcal{P}$ publishes its L1 signing public key(s) to the DROCA contract. The contract records these keys together with the party's role (asserter, watchtower, or both) and uses them to authenticate assertion submissions and dispute moves.

4.4 Phase 1

The Phase 1 has two stages: tournament registration and competition. The registration ends one timelock after it starts. When the registration is over, the registered participants are uniformly random sorted and the bracket tournament schedule is defined. The competition starts immediately after registration ends and schedule is ready.

4.4.1 Registration

All assertions must be registered before the Phase 1 tournament competition begins. If there were unresolved assertions queued for the next sequence number, then they are enqueued and automatically registered in the new Phase 1 tournament. The only new entrants that are accepted to a Phase 1 tournament are assertions having the next unresolved sequence number. All other received assertions are queued.

Once the tournament has one assertion included, a one-timelock-period window opens for other asserters to register conflicting sibling assertions. If no conflicts are registered within this window, the protocol bypasses Phase 1 and enters Phase 2 at the next timelock period. If one or more conflicting assertions are registered, Phase 1 begins at the next timelock period.

4.5 Phase 2

In Phase 2, the surviving assertion is open to challenge by any party.

4.5.1 Registration

A single timelock period is allocated for challenger registration. If no challenger registers, the surviving assertion is accepted. If challengers do register, they are uniformly randomly ordered and a bracket

schedule is instantiated in which each faces the surviving assenter. The assenter may initiate any eligible dispute immediately— either because a challenger has already timed out or because sufficient capital is available to run multiple disputes in parallel—but may not defer any dispute beyond its scheduled deadline.

4.5.2 Capital Use

In BATTLE, an honest assenter requires a *constant* minimum initial capital (independent of the number of challengers C) to win all disputes, and the worst-case number of rounds satisfies $R = \mathcal{O}(\log C)$. A comparable constant-capital guarantee holds for challengers provided the system enforces a bound on the number of *simultaneously active tournaments*. Enforcing such a concurrency cap is particularly challenging on UTXO-based blockchains that lack a reliable clock oracle, stateful scheduling primitives, or robust block-time introspection. In the absence of any cap, a challenger that aims to contest all active assertions must provision capital that grows linearly with the number of simultaneous tournaments.

We define the Cost of Dispute (A_{CoD}) as the maximum cost required to win a dispute in the foreseen future. It includes the amount spent on transaction fees and the cost of off-chain resources, such as computation and bandwidth, but does not include security bonds. We define the On-demand Security Bond (A_{OSB}) as the security deposit that each party posts before engaging in a two-party dispute. The Persistent Security Bond (A_{PSB}) is defined as a one-time lasting bond that exists to provide incentives other than to reward the winner of the dispute. The A_{PSB} must be held at least during a tournament. For example, if a party loses a dispute, a percentage of A_{PSB} can be burned, distributed among all honest operators evenly, or given to the first party to defeat that party. We define the Dispute Reward (A_{DR}) as the amount given to a party that wins a dispute, and it is taken from the losing party's A_{OSB} . The A_{DR} must be equal to A_{OSB} for a party to be able to reinvest the rewards to pay for future disputes. Other rewards can be paid from the A_{PSB} of the losing parties.

We also define the Minimum Initial Capital (A_{MIC}) as the minimum amount of funds that an honest party must initially hold to win all future disputes provided that it does not borrow funds. The A_{MIC} includes the A_{OSB} and A_{PSB} , but must also include the A_{CoD} at least once, which must be spent dynamically and is not part of the deposit.

In this work, we define A_{OSB} and A_{PSB} as the minimum amounts for initial capital to be sufficient to withstand disputes, but protocol designers may choose much higher amounts: the higher the security bonds, the lower the expected attack profit, which depends on the cost of attack failure multiplied by its probability.

Figure 3 shows the evolution of the wallet of a participant in a BATTLE tournament.

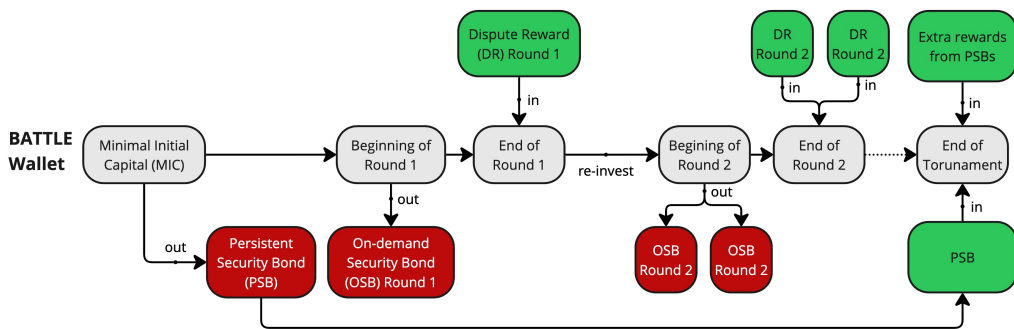


Figure 3: The evolution of a BATTLE wallet during a tournament

The BATTLE protocol differs from other protocols, such as BoLD or Dave, because it is designed to reuse the security bonds won in previous rounds, whereas the other protocols are not.

4.5.3 Tournament Schedules

BATTLE packs assertions and challenges into a tournament. All sibling assertions are synchronized in the sense that the protocol tries to pack most of them (honest and dishonest) within the same tournament. If the honest assertion for a certain sequence number is not yet available (i.e. it requires proving a blockchain event that has not been emitted), but a tournament of sibling assertions has started, then the honest assertion can be posted, but it is queued until the tournament is over.

If there are competing sibling assertions, the protocol starts with a phase resembling a bracket tournament where all the asserters register and compete against each other until only one wins.

In Phase 1, the asserters are paired, the first participant adopts the role ofasserter, and the second adopts the role of challenger. Which of the two acts as an asserter is predetermined. Therefore, the pair will dispute the assertion of one, while in that round the assertion of the other remains unchallenged. Figure 4 illustrates how the two phases are combined. The arrows in the Phase 1 illustration indicate the parties that will engage in a dispute, while the arrows in the Phase 2 illustration indicate how a single asserter funds are used and split to engage in disputes with multiple challenges.

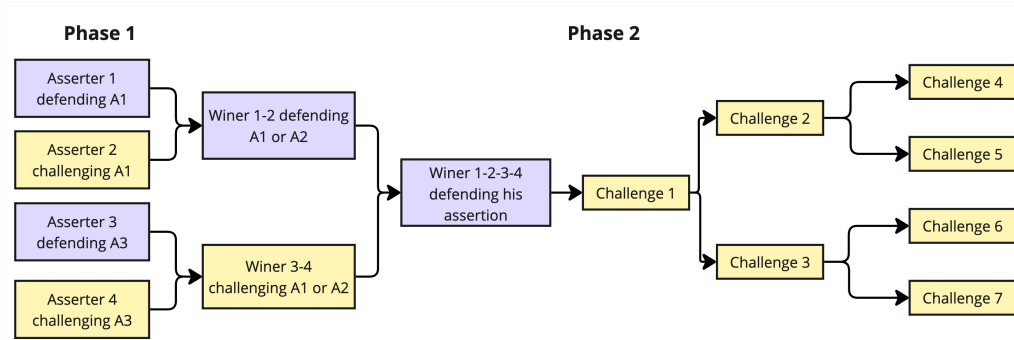


Figure 4: The two phases of the BATTLE protocol

For Phase 1, set $A_{PSB} = 0$, $A_{OSB} = A_{CoD}$, and $A_{MIC} = 2A_{CoD}$. With this choice, an honest asserter's initial capital is bounded by $2A_{CoD}$. In each round the asserter expends A_{CoD} in fees and, upon winning, receives a dispute reward $A_{DR} = A_{OSB} = A_{CoD}$ taken from the opponent's on-demand bond. The round is thus cash-neutral and preserves the A_{MIC} balance, enabling the asserter to advance through the bracket until a single survivor remains.

For Phase 2, we give four scheduling variants, with different trade-offs between initial capital and the worst-case number of rounds. All phase 2 tournaments require a A_{MIC} higher than the one required by phase 1, so the overall A_{MIC} for the protocol will be set based on the variant used for phase 2.

In all variants of Phase 2 we schedule disputes in the following way: All watchtowers that want to challenge must indicate so and post their security bond in the first timelock period after the Phase 2 starts. However, we let the asserter choose the order to fight back the challengers. The asserter has a different deadline to post his security bond for each dispute, and deadlines are established progressively until the tournament finalization time.

Disputes deadlines can be arranged statically or dynamically in a schedule. A static order is one that cannot depend on which challengers have registered in the tournament, while a dynamic order can. To produce the schedule, a random order of challengers is created. If the order is static, it will contain all potential challengers. If dynamic, only the challenges that have registered. Once we have the order, the protocol will attempt to allocate worst-case spots to them in rounds.

Now we present the Phase 2 schedule variants. To compare them, let R be a bound on the resulting

worst-case number of rounds of the Phase 2 protocol (any assertion is resolved in no more than R rounds).

4.5.4 Schedule 1: “maintain-or-double”

In this schedule variant, we can take $R_1 = 3 \cdot \log_2(C/3)$. In each round, the asserter attempts to double the number of disputes per round. If he cannot double the number, it maintains the same number of disputes in the next round.

We set $A_{\text{OSB}}^1 = 2A_{\text{CoD}}$, $A_{\text{DR}}^1 = A_{\text{OSB}}^1$, and $A_{\text{MIC}}^1 = 3A_{\text{CoD}}$. Note that the protocol doubles the number of disputes when the honest operator has collected enough funds to do it without additional capital inflows, and if that is not possible, another round combating the same number of challengers occurs.

We show that the number of disputes engaged per round is exactly $2^{\lceil r/3 \rceil}$ (i.e. 1,1,1, 2, 2, 2, 4, 4, 4, etc). This is because after winning the disputes of a round, the winner receives 1/3rd of the cost required to double the number of disputes. Even if the speed is slowed by a factor of 3 compared to continuous doubling, the maximum number of rounds is still $\mathcal{O}(\log(C))$.

Figure 5 shows this variant of Phase 2 of the BATTLE protocol.

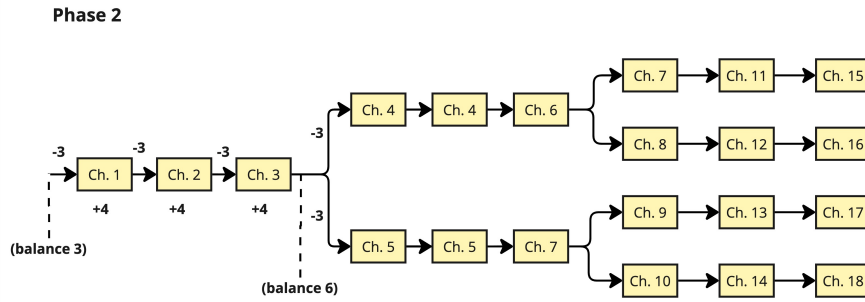


Figure 5: The BATTLE Phase 2 (variant 1)

4.5.5 Schedule 2: “always-double”

Now we attempt to double the number of disputes per round. We define two different values for $A_{\text{OSB}}^{\text{C}}$, $A_{\text{OSB}}^{\text{A}}$ (asserter deposit) and $A_{\text{OSB}}^{\text{C}}$ (challenger deposit). We set $A_{\text{OSB}}^{\text{A}} = 2A_{\text{CoD}}$ and $A_{\text{OSB}}^{\text{C}} = 4A_{\text{CoD}}$. We divide A_{DR}^{A} into A_{DR}^{A} and A_{DR}^{C} and $A_{\text{MIC}}^{\text{A}}$ into $A_{\text{MIC}}^{\text{A}}$ and $A_{\text{MIC}}^{\text{C}}$, and assign the values $A_{\text{DR}}^{\text{A}} = A_{\text{OSB}}^{\text{A}}$ and $A_{\text{DR}}^{\text{C}} = A_{\text{OSB}}^{\text{C}}$, $A_{\text{MIC}}^{\text{A}} = 3A_{\text{CoD}}$, and $A_{\text{MIC}}^{\text{C}} = 5A_{\text{CoD}}$. Now the amount won by an asserter on a dispute round is enough to pay for the cost of 2 following disputes (including its on-demand security bond and transaction fees) and we can set $R_2 = \log_2(C)$.

We note that the reward received by a winning challenger (A_{DR}^{C}) is not enough to cover the cost of engaging in twice as many disputes.

Figure 6 shows this variant of Phase 2 of the BATTLE protocol.

4.5.6 Schedule 3: Double Every Two Rounds

To avoid using asymmetric security bonds, we double the round size every two rounds. We set $A_{\text{OSB}}^3 = 3A_{\text{CoD}}$, $A_{\text{DR}}^3 = A_{\text{OSB}}^3$, and $A_{\text{MIC}}^3 = 4A_{\text{CoD}}$. We set $R_3 = 2 \log_2(C/2)$.

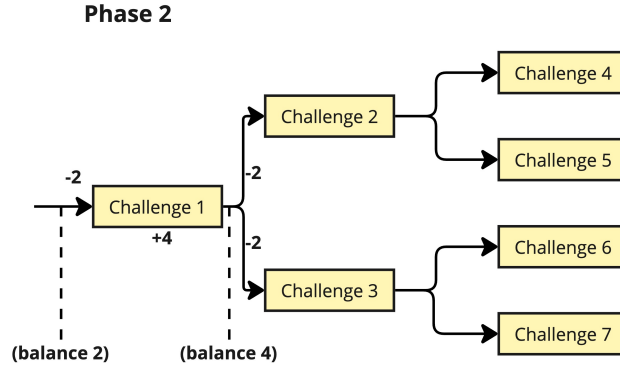


Figure 6: The BATTLE Phase 2 (variant 2). Asserters and Challengers post different amounts of security bonds.

Figure 7 shows the Phase 2 of the BATTLE protocol for this variant.

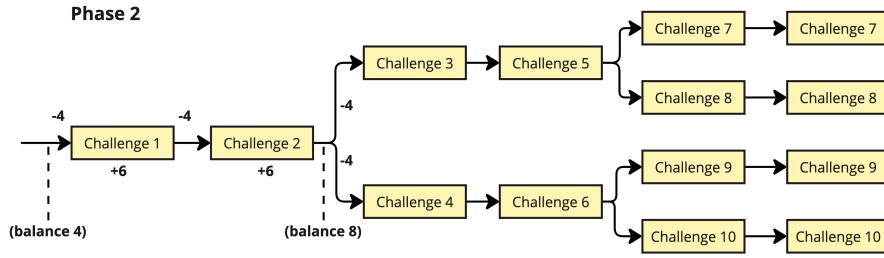


Figure 7: The BATTLE Phase 2 (variant 3)

4.5.7 Schedule 4: Gradual Increase

We can do better than all the previous schedules by re-using the revenue won at each round to engage in the maximum number of simultaneous disputes. We start with $A_{\text{OSB}}^4 = 2A_{\text{CoD}}$, $A_{\text{DR}}^4 = A_{\text{OSB}}^4$ and $A_{\text{MIC}}^4 = 3A_{\text{CoD}}$, the same values as defined in protocol schedule 1. We attempt a fine-grained increase in disputes per round, and the resulting progression of disputes per round begins with: 1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 22, 29, 39, 52, 69, 92. In round 12 it already doubles the number of simultaneous disputes reached by the maintain-or-double method, which means that our gradual protocol requires fewer rounds than in protocol variant 1. For $C < 1,000,000$, we can have $R_4 = 0.95 \log_{(4/3)}(C/0.95)$.

We can start this schedule at any round, setting A_{MIC}^4 high enough to pay for the first round. For example, setting $A_{\text{MIC}}^4 = 7A_{\text{CoD}}$, we start the tournament with the fifth term of the an sequence, having two disputes, to continue with 3,4,5,7, etc.

5 Summary

We isolate the dispute plane of optimistic systems via the DROCA abstraction: parties post parent-linked assertions, a contract evaluates them with a bounded-time verification predicate, and outcomes are accepted unless rejected within a challenge window. The model captures resource costs (gas, computation) and capital (A_{OSB} , A_{PSB}), defines a cost-of-dispute and a minimum initial capital, and operates in epochs with explicit rate controls (GAR/MGR) to prevent adversarial assertion growth. Within this setting, the dominant threat is resource exhaustion through Sybil-driven, concurrent disputes.

We introduce *BATTLE*, a two-phase tournament that recycles dispute rewards to fund later rounds, keeping the honest asserter’s initial capital constant while resolving C challenges in logarithmic rounds. Phase 1 eliminates competing asserters in a bracket; Phase 2 escalates the number of simultaneous disputes per round according to schedules that trade rounds for liquidity, with both symmetric and asymmetric bond options. The approach preserves decentralization by allowing a high number of asserters and challengers, converts adversarial breadth into time rather than cash, and attains constant-capital progress for asserters; constant-capital guarantees for challengers hold when the system bounds the number of simultaneously active tournaments.

In sum, *BATTLE* provides a concrete path to DoS-resilient, constant-capital dispute resolution that retains openness while delivering logarithmic settlement time.

References

- [1] J. Teutsch and C. Reitwießner, “A scalable verification solution for blockchains,” 2017. [Online]. Available: <https://arxiv.org/abs/1908.04756v1>
- [2] S. Jain, P. Saxena, F. Stephan, and J. Teutsch, “How to verify computation with a rational network,” 2016, accessed: 2025-07-26. [Online]. Available: <https://arxiv.org/abs/1606.05917>
- [3] S. Lee, “Hollow victory: How malicious proposers exploit validator incentives in optimistic rollup dispute games,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.05094>
- [4] P. McCorry, C. Buckland, B. Yee, and D. Song, “Sok: Validating bridges as a scaling solution for blockchains,” Cryptology ePrint Archive, Report 2021/1589, 2021, <https://ia.cr/2021/1589>.
- [5] D. L. Fekete and A. Kiss, “Trust-minimized optimistic cross-rollup arbitrary message bridge,” *Journal of Network and Computer Applications*, vol. 221, p. 103771, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S108480452300190X>
- [6] M. Zhang, X. Zhang, J. Barbee, Y. Zhang, and Z. Lin, “Sok: Security of cross-chain bridges: Attack surfaces, defenses, and open problems,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.12573>
- [7] S. Avizheh, M. Nabi, and R. Safavi-Naini, “Refereed delegation of computation using smart contracts,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5208–5227, 2024.
- [8] D. Nehab and A. Teixeira, “Permissionless refereed tournaments,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.12439>
- [9] M. M. Alvarez, H. Arneson, B. Berger, L. Bousfield, C. Buckland, Y. Edelman, E. W. Felten, D. Goldman, R. Jordan, M. Kelkar, A. Mamageishvili, H. Ng, A. Sanghi, V. Shoup, and T. Tsao, “Bold: Fast and cheap dispute resolution,” 2024, accessed: 2025-07-26. [Online]. Available: <https://arxiv.org/abs/2404.10491>
- [10] D. Nehab, G. C. de Paula, and A. Teixeira, “Dave: a decentralized, secure, and lively fraud-proof algorithm,” 2024, accessed: 2025-07-26. [Online]. Available: <https://arxiv.org/abs/2411.05463>
- [11] Optimism Foundation. Fault proofs explainer. OP Stack Documentation. Optimism docs page describing the OP Stack fault/fraud-proof mechanism. [Online]. Available: <https://docs.optimism.io/stack/fault-proofs/explainer>
- [12] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” <https://ethereum.github.io/yellowpaper/paper.pdf>, 2014, ethereum Project Yellow Paper.

- [13] S. D. Lerner, J. Álvarez Cid-Fuentes, J. Len, R. Fernández-València, P. Gallardo, N. Vescovo, R. Laprida, S. Mishra, F. Jinich, and D. Masini, “Rsk: A bitcoin sidechain with stateful smart contracts,” Cryptology ePrint Archive, Paper 2022/684, 2022, accessed: 2025-07-26. [Online]. Available: <https://eprint.iacr.org/2022/684>
- [14] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Proceedings of the 2016 ACM Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 305–317. [Online]. Available: <https://dl.acm.org/doi/10.1145/2976749.2978318>
- [15] G. C. de Paula. (2024, April 8) Fraud proofs are broken. Ethereum Research forum post. [Online]. Available: <https://ethresear.ch/t/fraud-proofs-are-broken/19234>
- [16] J. Teutsch and C. Reitwießner, “A scalable verification solution for blockchains,” 2019. [Online]. Available: <https://arxiv.org/abs/1908.04756>
- [17] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, “Arbitrum: Scalable, private smart contracts,” in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 1353–1370. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner>